

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
“НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО”

Факультет	Программной Инженерии и Компьютерной Техники
Направление подготовки	
(специальность)	
Дисциплина	Тестирование программного обеспечения

ЛАБОРАТОРНАЯ РАБОТА 4

Выполнил студент: Савин Денис 409494
Группа: Р3369
Преподаватель: Наумова Надежда Александровна

г. Санкт-Петербург

2026

Описание задания

С помощью программного пакета Apache JMeter провести нагрузочное и стресс-тестирование веб-приложения в соответствии с вариантом задания.

В ходе нагрузочного тестирования необходимо протестировать 3 конфигурации аппаратного обеспечения и выбрать среди них наиболее дешёвую, удовлетворяющую требованиям по максимальному времени отклика приложения при заданной нагрузке (в соответствии с вариантом).

В ходе стресс-тестирования необходимо определить, при какой нагрузке выбранная на предыдущем шаге конфигурация перестает удовлетворять требованиям по максимальному времени отклика. Для этого необходимо построить график зависимости времени отклика приложения от нагрузки.

Приложение для тестирования доступно только во внутренней сети кафедры.

Если запрос содержит некорректные параметры, сервер возвращает HTTP 403.

Если приложение не справляется с нагрузкой, сервер возвращает HTTP 503.

Параметры тестируемого веб-приложения:

Параметры тестируемого веб-приложения:

- URL первой конфигурации (\$ 4600) - <http://stload.se.ifmo.ru:8080?token=518502949&user=-1355159875&config=1;>
- URL второй конфигурации (\$ 5400) - <http://stload.se.ifmo.ru:8080?token=518502949&user=-1355159875&config=2;>
- URL третьей конфигурации (\$ 7600) - <http://stload.se.ifmo.ru:8080?token=518502949&user=-1355159875&config=3;>
- Максимальное количество параллельных пользователей - 10;
- Средняя нагрузка, формируемая одним пользователем - 40 запр. в мин.;
- Максимально допустимое время обработки запроса - 600 мс.

Выполнение

Исходный код

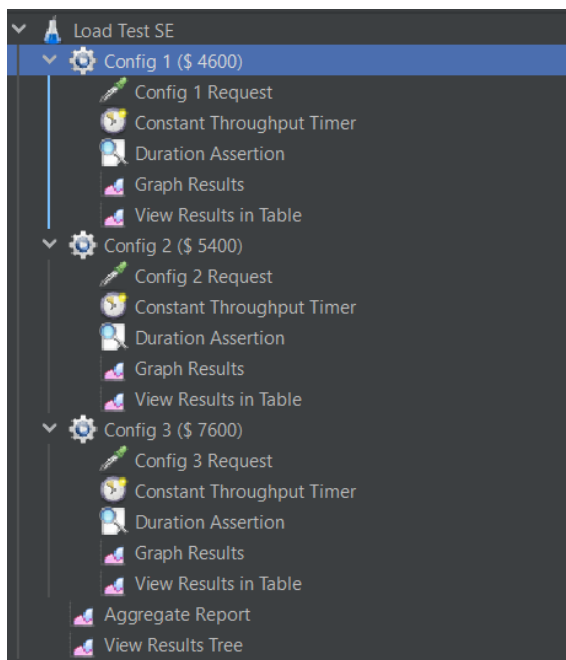
https://github.com/Undefined/Software_Testing_2026/tree/main/Lab4

Описание конфигурации JMeter для нагрузочного тестирования

Тестовый сценарий состоит из 3 конфигураций: Config 1 (\$ 4600), Config 2 (\$ 5400) и Config 3 (\$ 7600) — соответственно для каждой конфигурации веб-приложения.

Общая конфигурация

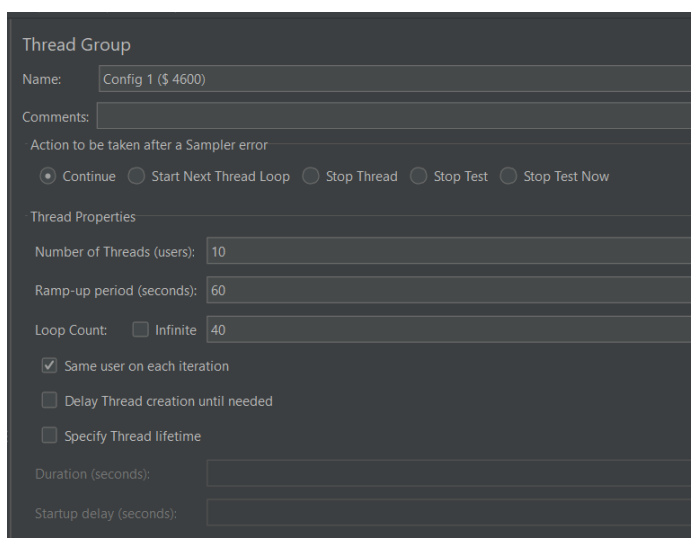
Конфигурация нагрузочного тестирования состоит из 3 объектов Thread Group, по одному на каждую конфигурацию; конфигурация каждого из них идентична, за исключением параметра config при формировании запроса. Помимо Thread Group присутствуют вспомогательные элементы для генерации общих отчётов (Aggregate Report, Aggregate Graph).



Конфигурация Thread Group

Параметр	Значение
Number of Threads (users)	10
Ramp-up period (seconds)	60
Loop Count	40
Same user on each iteration	включено
Action to be taken after a Sampler error	Continue

Значения Number of Threads (10) и Loop Count (40) взяты из варианта. Ramp-up period взят по умолчанию — 60 секунд, JMeter постепенно доводит число активных потоков с 0 до 10.



HTTP Request

Осуществляет запросы к тестовому серверу. Параметры (token, user, config) заданы через табличное представление в разделе Parameters, а не напрямую через URL.

The screenshot shows the 'HTTP Request' configuration window. It has a 'Name' field set to 'Config 1 Request' and an empty 'Comments' field. Below are tabs for 'Basic' and 'Advanced'. The 'Basic' tab is active, showing 'Web Server' settings: Protocol (http), Server Name or IP (127.0.0.1), and Port Number (8079). The 'HTTP Request' section shows the Method as 'GET' and Path as '/'. There are checkboxes for 'Redirect Automatically' (unchecked), 'Follow Redirects' (checked), 'Use KeepAlive' (checked), 'Use multipart/form-data' (unchecked), and 'Browser-compatible headers' (unchecked). Below this is the 'Parameters' tab, which is active. It shows a table for 'Send Parameters With the Request'.

Name:	Value	URL Encode?	Content-Type
token	518502949	☐	text/plain
user	-1355159875	☐	text/plain
config	1	☐	text/plain

В зависимости от конфигурации меняется только config

Duration Assertion

Проверяет время отклика сервера: запрос считается неуспешным, если пороговое время выполнения превышено. Для варианта — 600 мс.

The screenshot shows the 'Duration Assertion' configuration window. It has a 'Name' field set to 'Duration Assertion' and an empty 'Comments' field. Below is the 'Apply to:' section with three radio buttons: 'Main sample and sub-samples' (unchecked), 'Main sample only' (checked), and 'Sub-samples only' (unchecked). Below this is the 'Duration to Assert' section with a 'Duration in milliseconds:' field set to '600'.

Constant Throughput Timer

Определяет среднюю нагрузку, формируемую одним пользователем в минуту. Для варианта ограничено 40 запросами в минуту (calcMode = this thread only).

Constant Throughput Timer

Name:

Comments:

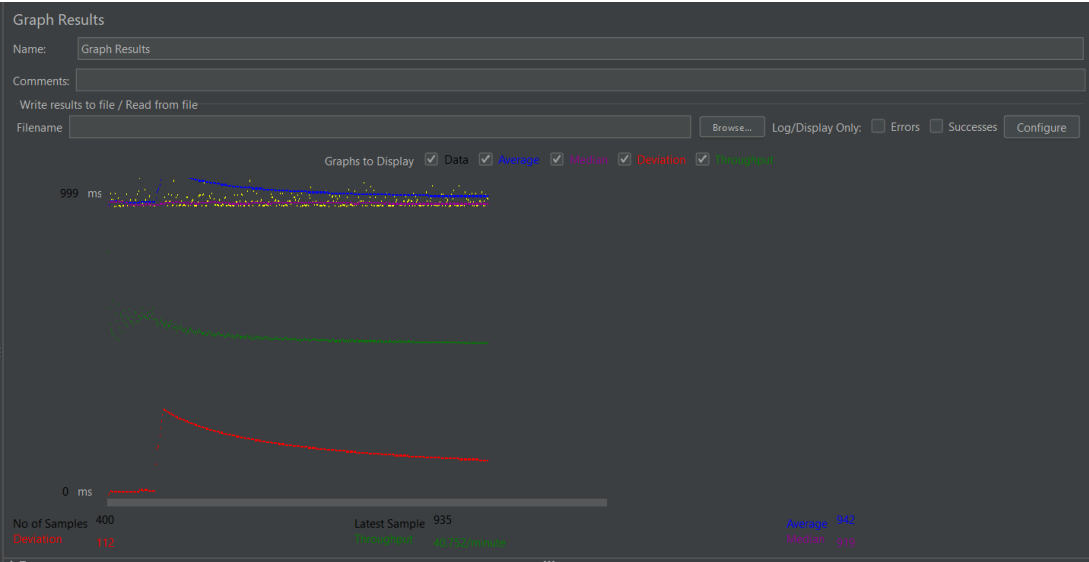
☐ Delay before each affected sampler

Target throughput (in samples per minute):

Calculate Throughput based on:

Графики и таблицы пропускной способности приложения, полученные в ходе нагрузочного тестирования

Конфигурация 1



View Results in Table

Name:

Comments:

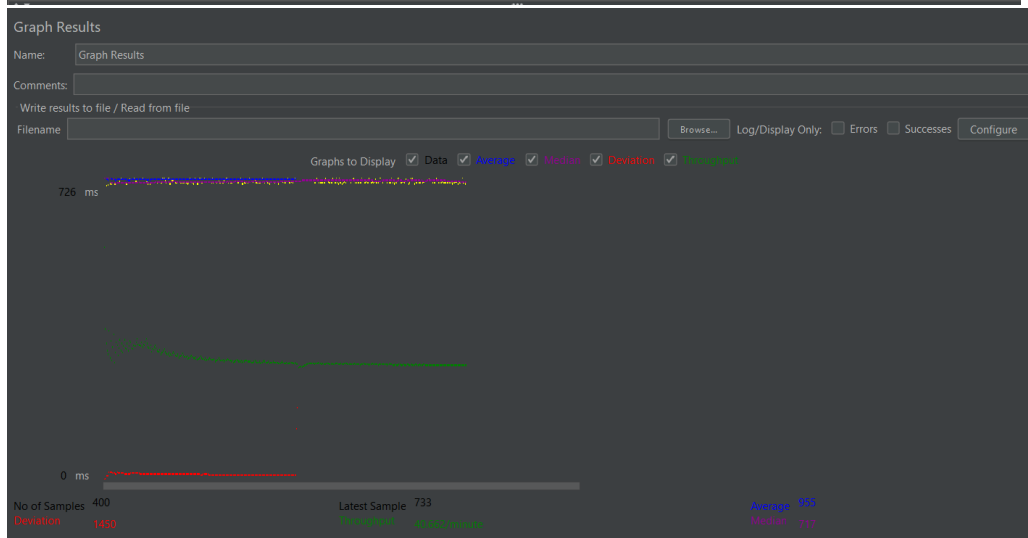
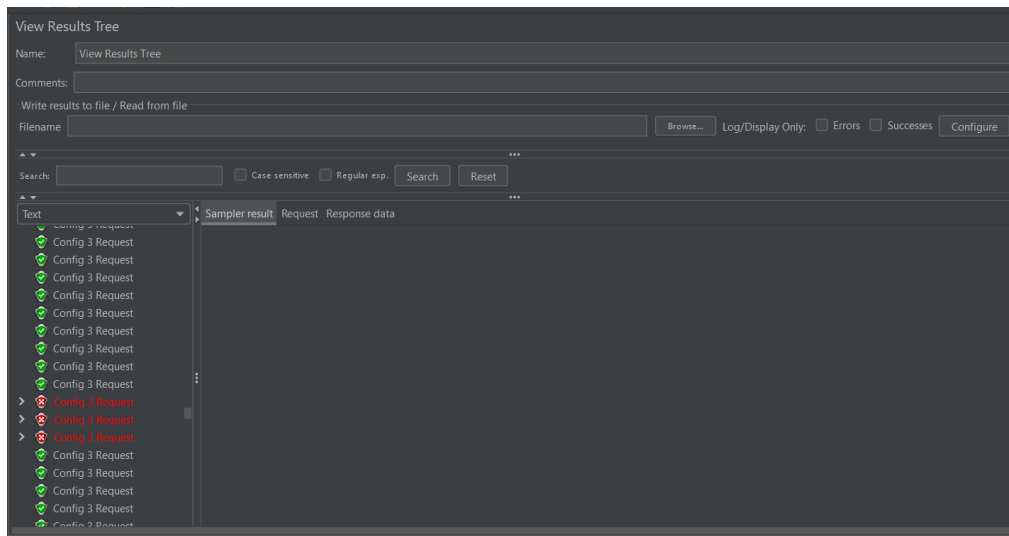
Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

Sample #	Start Time	Thread Name	Label	Sample Time(ms)	Status	Bytes	Sent Bytes	Latency	Connect Time(ms)
1	13:09:44.194	Config 1 (\$ 4600) ...	Config 1 Request	924		231	158	924	1
2	13:09:45.693	Config 1 (\$ 4600) ...	Config 1 Request	909		231	158	909	0
3	13:09:47.193	Config 1 (\$ 4600) ...	Config 1 Request	951		231	158	951	0
4	13:09:48.694	Config 1 (\$ 4600) ...	Config 1 Request	932		231	158	932	0
5	13:09:50.192	Config 1 (\$ 4600) ...	Config 1 Request	925		231	158	925	0
6	13:09:50.191	Config 1 (\$ 4600) ...	Config 1 Request	931		231	158	931	1
7	13:09:53.191	Config 1 (\$ 4600) ...	Config 1 Request	911		231	158	911	0
8	13:09:53.192	Config 1 (\$ 4600) ...	Config 1 Request	951		231	158	951	0
9	13:09:56.191	Config 1 (\$ 4600) ...	Config 1 Request	911		231	158	911	0
10	13:09:56.193	Config 1 (\$ 4600) ...	Config 1 Request	929		231	158	929	0
11	13:09:56.191	Config 1 (\$ 4600) ...	Config 1 Request	936		231	158	936	1
12	13:10:00.691	Config 1 (\$ 4600) ...	Config 1 Request	912		231	158	912	0
13	13:10:00.691	Config 1 (\$ 4600) ...	Config 1 Request	912		231	158	912	0
14	13:10:00.693	Config 1 (\$ 4600) ...	Config 1 Request	911		231	158	911	0
15	13:10:02.192	Config 1 (\$ 4600) ...	Config 1 Request	928		231	158	928	1
16	13:10:05.190	Config 1 (\$ 4600) ...	Config 1 Request	913		231	158	913	0
17	13:10:05.190	Config 1 (\$ 4600) ...	Config 1 Request	913		231	158	913	0
18	13:10:05.193	Config 1 (\$ 4600) ...	Config 1 Request	913		231	158	913	0
19	13:10:08.191	Config 1 (\$ 4600) ...	Config 1 Request	923		231	158	923	0
20	13:10:08.190	Config 1 (\$ 4600) ...	Config 1 Request	940		231	158	940	0

[illegible]

Конфигурация 2



View Results in Table

Name:

View Results in Table

Comments:

Write results to file / Read from file

Filename

Browse...

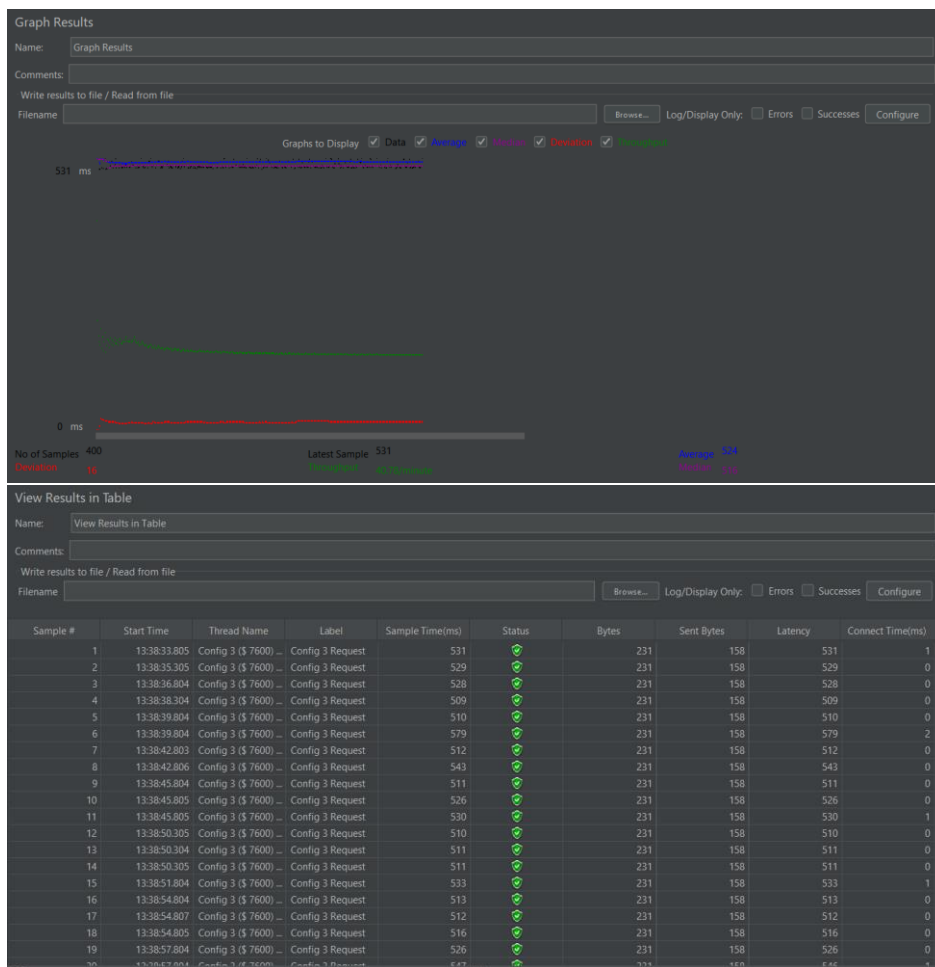
Log/Display Only:

☐ Errors
 ☐ Successes

Configure

Sample #	Start Time	Thread Name	Label	Sample Time(ms)	Status	Bytes	Sent Bytes	Latency	Connect Time(ms)
1	13:25:04.564	Config 2 (\$ 5400) ...	Config 2 Request	726		231	158	726	2
2	13:25:06.063	Config 2 (\$ 5400) ...	Config 2 Request	733		231	158	733	0
3	13:25:07.563	Config 2 (\$ 5400) ...	Config 2 Request	708		231	158	708	0
4	13:25:09.063	Config 2 (\$ 5400) ...	Config 2 Request	727		231	158	727	0
5	13:25:10.562	Config 2 (\$ 5400) ...	Config 2 Request	710		231	158	710	0
6	13:25:10.562	Config 2 (\$ 5400) ...	Config 2 Request	768		231	158	768	1
7	13:25:13.563	Config 2 (\$ 5400) ...	Config 2 Request	711		231	158	711	0
8	13:25:13.563	Config 2 (\$ 5400) ...	Config 2 Request	713		231	158	713	0
9	13:25:16.563	Config 2 (\$ 5400) ...	Config 2 Request	711		231	158	711	0
10	13:25:16.565	Config 2 (\$ 5400) ...	Config 2 Request	723		231	158	723	0
11	13:25:16.563	Config 2 (\$ 5400) ...	Config 2 Request	732		231	158	732	2
12	13:25:21.063	Config 2 (\$ 5400) ...	Config 2 Request	711		231	158	711	0
13	13:25:21.063	Config 2 (\$ 5400) ...	Config 2 Request	711		231	158	711	0
14	13:25:21.065	Config 2 (\$ 5400) ...	Config 2 Request	711		231	158	711	0
15	13:25:22.561	Config 2 (\$ 5400) ...	Config 2 Request	761		231	158	761	1
16	13:25:25.562	Config 2 (\$ 5400) ...	Config 2 Request	715		231	158	715	0
17	13:25:25.564	Config 2 (\$ 5400) ...	Config 2 Request	714		231	158	714	0
18	13:25:25.561	Config 2 (\$ 5400) ...	Config 2 Request	719		231	158	719	0
19	13:25:28.562	Config 2 (\$ 5400) ...	Config 2 Request	724		231	158	724	0
20	13:25:28.562	Config 2 (\$ 5400) ...	Config 2 Request	744		231	158	744	0

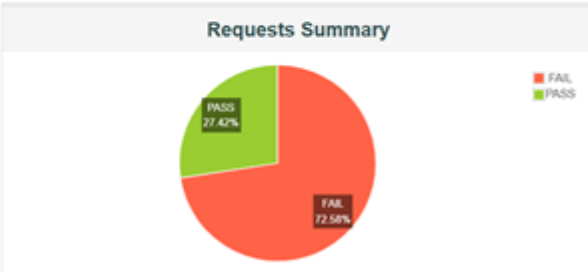
Конфигурация 3



За счет инструмента jmeter можно построить «не вырвиглазные» графики, и тем самым провел для всех 3 конфигураций нагрузочное тестирование одновременно и мы еще раз можем убедиться что конфигурация 3 является наилучшей

Test and Report information	
Source file	"load_results.jlr"
Start Time	"9/10/26, 6:06 PM"
End Time	"9/10/26, 6:35 PM"
Filter for display	**

APDEX (Application Performance Index)			
Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
0.137	500 ms	1 sec 500 ms	Total
0.000	500 ms	1 sec 500 ms	Config 1 Request
0.000	500 ms	1 sec 500 ms	Config 2 Request
0.411	500 ms	1 sec 500 ms	Config 3 Request



Statistics													
Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	1200	871	72.58%	1346.22	507	12557	757.00	2521.20	5484.60	10219.96	0.68	0.15	0.11
Config 1 Request	400	400	100.00%	1560.48	907	12557	950.50	2762.20	6025.75	10384.10	0.23	0.05	0.04
Config 2 Request	400	400	100.00%	1342.17	708	11575	747.00	2562.90	5476.00	10219.31	0.23	0.05	0.04
Config 3 Request	400	71	17.75%	1136.01	507	10367	546.00	2420.70	5450.30	9873.55	0.23	0.05	0.04

Выводы по выбранной конфигурации аппаратного обеспечения

В ходе нагрузочного тестирования сравнивались три конфигурации при 10 параллельных пользователей и интенсивности 40 HTTP-запросов в минуту на пользователя. Критерий соответствия — время обработки запроса не более 600 мс.

Конфигурация 1:

Label	# Sa...	Ave...	Me...	90...	95...	99...	Min	Max	Erro...	Thro...	Received K...	Sent K...
Config 1 Re...	400	942	919	952	965	1761	909	1847	100...	.67919	0.15	0.10
TOTAL	400	942	919	952	965	1761	909	1847	100...	.67919	0.15	0.10

Конфигурация 2:

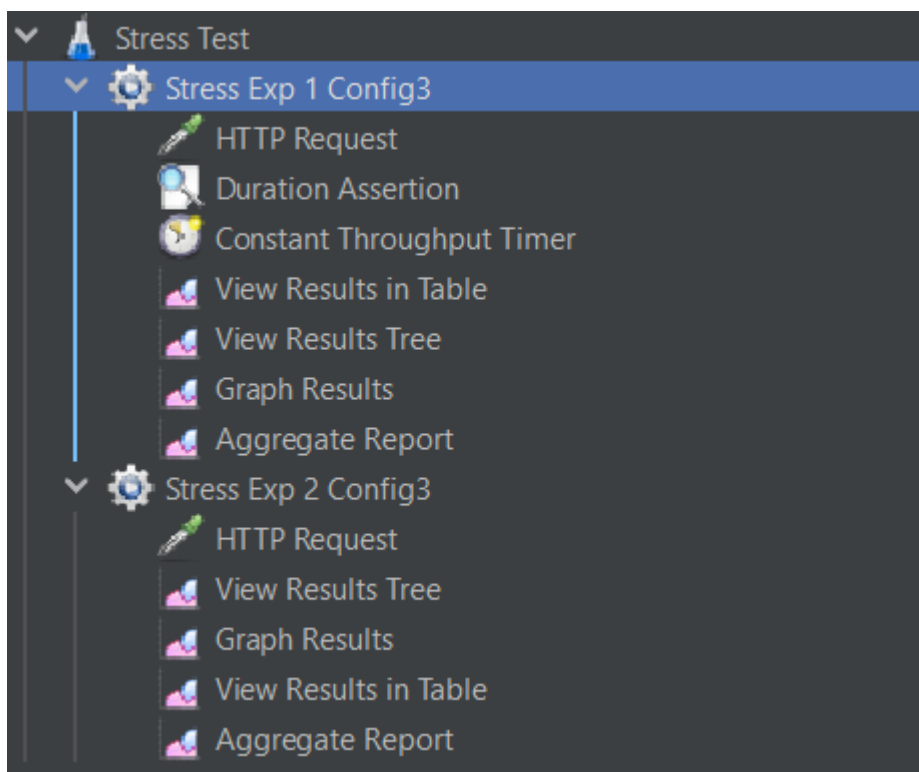
Label	# Sa...	Aver...	Med...	90% ...	95% ...	99% ...	Min	Max	Error...	Throu...	Received KB...	Sent K...
Config 2 Req...	400	955	717	746	774	10437	708	13426	100...	.67770	0.15	0.10
TOTAL	400	955	717	746	774	10437	708	13426	100...	.67770	0.15	0.10

Конфигурация 3:

Label	# Sa...	Aver...	Med...	90% ...	95% ...	99% ...	Min	Max	Error...	Throu...	Received KB...	Sent K...
Config 3 Req...	400	524	516	544	555	581	509	620	0.75...	.67966	0.15	0.10
TOTAL	400	524	516	544	555	581	509	620	0.75...	.67966	0.15	0.10

Вывод: оптимальной является конфигурация 3, так как она самая дешёвая среди удовлетворяющих требованиям по времени отклика при заданной нагрузке всего 0.75% ошибок

Описание конфигурации JMeter для стресс-тестирования



Проведены два эксперимента:

1. Определение количества пользователей, при котором сервис перестаёт укладываться в лимит 600 мс.
2. Нахождение точки, в которой сервис начинает отдавать HTTP 503 (предел пропускной способности).

Эксперимент 1

Число пользователей плавно наращивается (ramp-up 100 с, до 100 потоков, длительность 100 с), пока среднее время отклика не превысит лимит 600 мс.

Берем конфигурацию 3

Thread Group

Name: Stress Exp 1 Config3

Comments:

Action to be taken after a Sampler error

☒ Continue ☐ Start Next Thread Loop ☐ Stop Thread ☐ Stop Test ☐ Stop Test Now

Thread Properties

Number of Threads (users): 100

Ramp-up period (seconds): 100

Loop Count: ☒ Infinite

☒ Same user on each iteration

☐ Delay Thread creation until needed

☒ Specify Thread lifetime

Duration (seconds): 60

Startup delay (seconds):

HTTP Request

Name: HTTP Request

Comments:

^ v

...

Basic Advanced

Web Server

Protocol (http): http Server Name or IP: 127.0.0.1

HTTP Request

GET Path: /

☐ Redirect Automatically ☒ Follow Redirects ☒ Use KeepAlive ☐ Use multipart/form-data ☐ Browser-compatible headers

Parameters Body Data Files Upload

Send Parameters With the Request:

Name:	Value	URL Encode?
token	518502949	☐
user	-1355159875	☐
config	3	☐

Duration Assertion

Name: Duration Assertion

Comments:

Apply to:

☐ Main sample and sub-samples ☒ Main sample only ☐ Sub-samples only

Duration to Assert

Duration in milliseconds: 600

Constant Throughput Timer

Name: Constant Throughput Timer

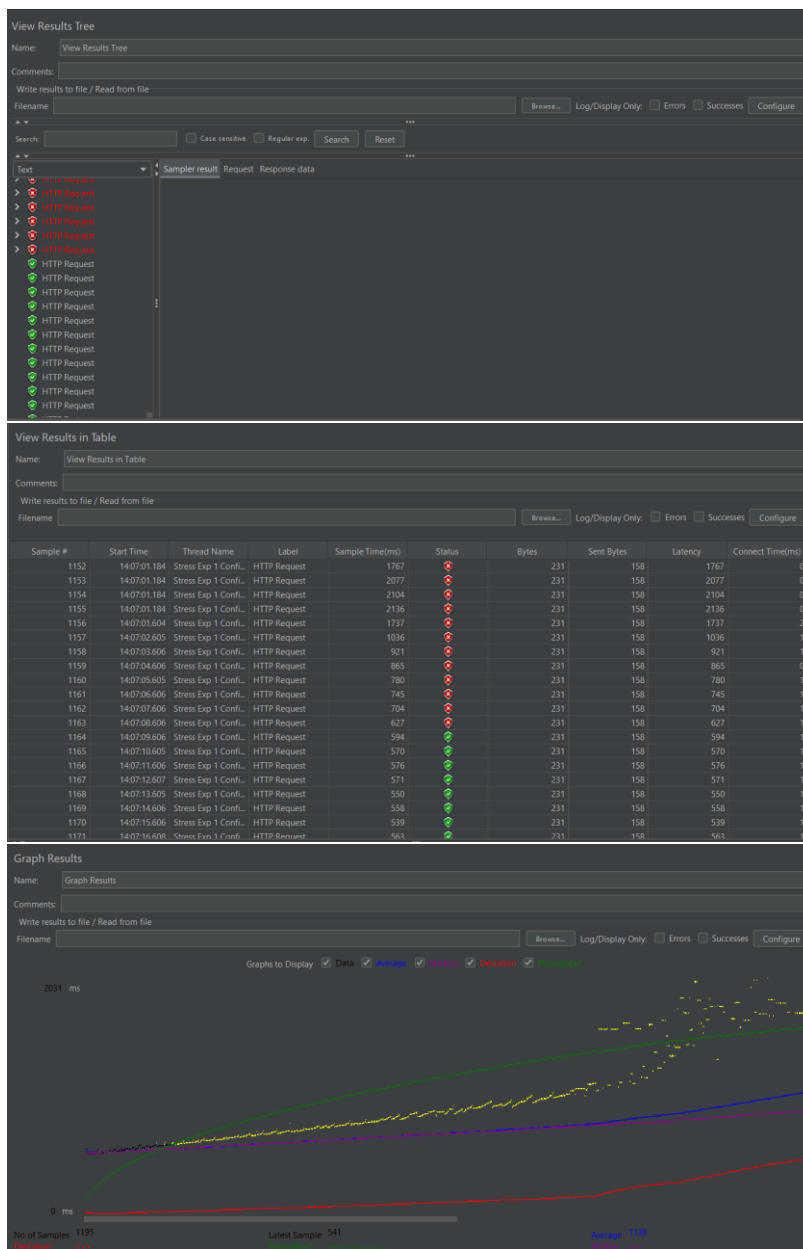
Comments:

- Delay before each affected sampler

Target throughput (in samples per minute): 40.0

Calculate Throughput based on: this thread only

И результаты:



Эксперимент 2

В итоге так и не получилось словить несчастную 503, предполагаю что это впринципе невозможно с моими параметрами

Далее привел таблицу где постепенно увеличивал нагрузку на сервер

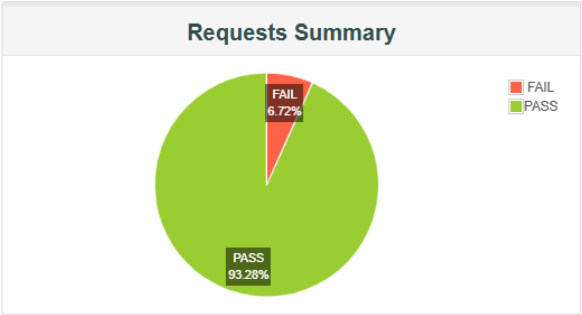
users	rpm	samples	http_200	http_503	other_http	sla_errors	error_rate	avg_ms	median_ms	p90_ms	p95_ms	p99_ms	max_ms	has_503	measured_throughput_rpm
1	20	6	6	0	0	0	0.00	535.33	517.50	581.50	600.25	615.25	619.00	false	24.10
2	40	61	61	0	0	1	1.64	528.46	530.00	541.00	548.00	598.00	628.00	false	40.94
3	60	92	92	0	0	4	4.35	546.51	515.00	560.20	598.65	1142.79	1444.00	false	61.63
4	80	123	123	0	0	3	2.44	547.34	536.00	549.60	575.70	1019.00	1286.00	false	82.33
5	100	154	154	0	0	0	0.00	531.31	520.00	555.70	579.35	593.87	605.00	false	103.00
7	140	215	215	0	0	6	2.79	542.04	526.00	560.60	582.20	775.82	1028.00	false	143.48
10	200	306	306	0	0	18	5.88	564.74	543.50	572.50	645.75	1139.35	1213.00	false	204.20
12	240	365	365	0	0	10	2.74	548.94	540.00	570.20	578.00	658.08	726.00	false	244.94
16	320	487	487	0	0	5	1.03	564.71	560.00	583.00	596.40	618.20	1196.00	false	325.01
20	400	603	603	0	0	26	4.31	582.10	570.00	599.80	618.00	913.14	1478.00	false	402.42
24	480	722	722	0	0	49	6.79	595.62	581.00	611.90	628.00	1052.85	1443.00	false	481.92
32	640	954	954	0	0	471	49.37	621.38	620.00	645.00	655.00	681.23	938.00	false	636.82
40	800	1185	1185	0	0	1080	91.14	659.51	662.00	686.00	693.00	735.28	1394.00	false	791.01
48	960	1405	1405	0	0	1299	92.46	707.38	712.00	736.00	748.00	903.04	1250.00	false	937.74
56	1120	1618	1618	0	0	1525	94.25	760.42	774.00	798.00	809.00	841.00	1227.00	false	1079.97
64	1280	1838	1838	0	0	1731	94.18	823.64	846.00	870.00	879.00	911.15	1208.00	false	1226.89
72	1440	2055	2055	0	0	1961	95.43	904.64	933.00	963.00	973.00	1075.46	1450.00	false	1371.66
80	1600	2276	2276	0	0	2177	95.65	998.17	1037.00	1079.00	1099.00	1179.25	1428.00	false	1520.63
96	1920	2495	2495	0	0	2395	95.99	2309.70	2390.00	3451.00	3635.30	3844.06	4533.00	false	1669.99
112	2240	2505	2505	0	0	2403	95.93	2733.61	2812.00	4266.00	4401.00	4535.96	5341.00	false	1687.15

В ходе стресс-тестирования НТТР 503 не был зафиксирован вплоть до 224 пользователей. Однако уже при нагрузке 32 пользователя (640 запросов/мин) среднее время ответа превысило 620 мс (621 мс), а доля ошибок достигла 49%. При нагрузке 40 пользователей (800 запросов/мин) среднее время ответа составило примерно 660 мс, а доля ошибок — 91%. Это свидетельствует о превышении предельной нагрузки. Сервер возвращал ошибки, отличные от 503 (вероятно, НТТР 500 или таймауты), что может быть связано с особенностями конфигурации приложения

Графики для выбранной конфигурации

Test and Report information	
Source file	"stress_results.jtl"
Start Time	"9/10/26, 9:13 PM"
End Time	"9/10/26, 9:16 PM"
Filter for display	""

APDEX (Application Performance Index)			
Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
0.025	500 ms	1 sec 500 ms	Total
0.025	500 ms	1 sec 500 ms	HTTP Request



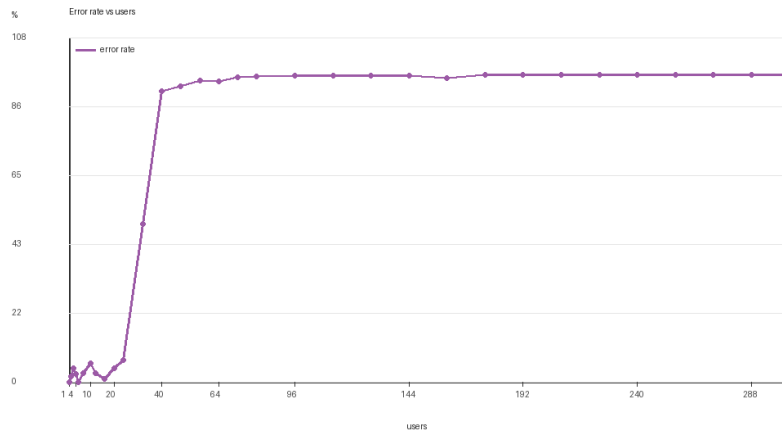
Statistics

Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label ^	#Samples ^	FAIL ^	Error % ^	Average ^	Min ^	Max ^	Median ^	90th pct ^	95th pct ^	99th pct ^	Transactions/s ^	Received ^	Sent ^
Total	5359	360	6.72%	8836.70	519	27532	8370.00	16369.00	19968.00	24426.40	28.12	6.35	4.34
HTTP Request	5359	360	6.72%	8836.70	519	27532	8370.00	16369.00	19968.00	24426.40	28.12	6.35	4.34

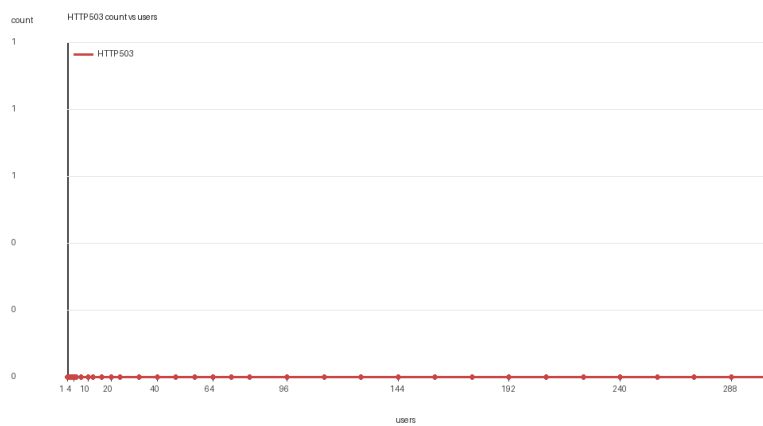
Errors			
Type of error	Number of errors	% in errors	% in all samples
500/Internal Server Error	360	100.00%	6.72%

Были написаны скрипты на Python для автоматизации стресс-тестирования: генерация JMX-планов, запуск JMeter, анализ JTL-файлов и построение графиков

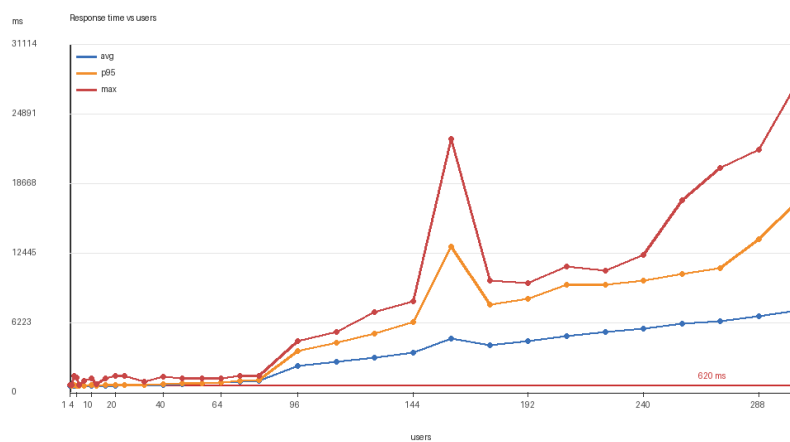
Количество ошибок по увеличению юзеров:



Количество 503 ошибок:



Время отклика:



Выводы по работе

В ходе выполнения лабораторной работы были изучены методы нагрузочного и стресс-тестирования веб-приложений с использованием программного пакета Apache JMeter. Были настроены сценарии тестирования, выполнена генерация нагрузки и проведён анализ основных показателей производительности системы.